Beyond Solution Dumping: Training Pedagogical Coding Tutors via Verifiable RL and Isomorphic Transfer Rewards

Chau The Vi
GStar Bootcamp 2026 — Individual Project Proposal

Track: Post-Training (RL), Synthetic Environments & Human–AI Interaction

1. Problem Description & Limitations

Standard Large Language Models (LLMs) post-trained with conventional Reinforcement Learning from Human Feedback (RLHF) or instruction tuning are optimized to maximize immediate user satisfaction and task completion. While effective for pure automation, this objective is fundamentally misaligned with human skill acquisition. As highlighted in recent human–AI interaction discussions, **a model that solves a problem well often actively harms human learning** by inducing cognitive offloading. In computer science education and algorithmic debugging, frontier LLMs exhibit three critical failure modes when deployed as educational tutors:

1. **Premature Solution Leakage:** Upon inspecting a buggy student program, models immediately output corrected code blocks or explicit line-by-line patches, depriving learners of the diagnostic struggle required for conceptual retention.
2. **Sycophantic & Shallow Hints:** When prompted to "act as a Socratic tutor," models rely on brittle prompt adherence—frequently collapsing into giving away the answer in natural language disguise when a student expresses frustration across turns.
3. **Lack of Pedagogical Optimization:** Existing post-training pipelines evaluate single-turn response helpfulness rather than **downstream human learning gain** (i.e., whether the tutoring guidance equips the learner to solve subsequent unseen problems independently).

Project Goal: We propose to develop **SocraticCoder-7B**, a specialized pedagogical coding tutor trained via Reinforcement Learning with Verifiable Rewards (RLVR). Instead of optimizing solely for task resolution on the current problem, we train the tutor policy to maximize a simulated student's **zero-shot transfer performance on held-out isomorphic problems** under strict, verifiable non-leakage constraints.

2. Technical Proposal

2.1 Technical Approach

To ensure high training throughput and 6-week feasibility without the instability of long-horizon online multi-agent rollouts, we formulate pedagogical tutoring via **Contextual State-Jump RL** between a trainable **Tutor Policy** (π_θ, initialized from `Qwen2.5-Coder-7B-Instruct` with LoRA/full-FT) and a fast, frozen **Simulated Student** (π_{stu}, initialized from an instruction-tuned 3B/7B model conditioned on explicit misconception personas). The end-to-end framework (Figure 1) consists of three core stages:

Stage 1: Paired Isomorphic Problem & Misconception Environment. Building on the paradigm of pairing synthetic dialogues with executable checkers (*EigenData*), we curate a focused, high-quality dataset of **1,200 paired coding tasks** (P, P') (1,000 train, 200 test) derived from *HumanEvalFix* and *MBPP+*:

- **Practice State (s_t on P):** A coding task P, a buggy student submission c_0 embodying a specific algorithmic flaw (e.g., off-by-one DP boundary, missing base case), and a pre-sampled dialogue history h_t ($1 \le t \le 3$ turns) where the student expresses confusion or requests a hint.
- **Isomorphic Transfer Problem (P'):** A structurally analogous problem generated via local LLM synthesis (`Qwen2.5-32B`) requiring the exact same underlying algorithmic principle, verified via deterministic unit tests $U(P')$.

Stage 2: Contextual Tutor Intervention & Zero-Shot Transfer Verification. During RL training, given state s_t, Tutor π_θ generates a group of $G = 8$ candidate pedagogical responses $\{a_t^{(i)}\}_{i=1}^{G}$. For each response a_t, the frozen Student π_{stu} first attempts to fix its buggy code on P (producing c_{final}). Immediately after, we evaluate **Cognitive Transfer**: we condition π_{stu} on the updated interaction $(s_t, a_t, c_{\text{final}})$ and prompt it to solve the unseen isomorphic problem P' **zero-shot without tutor intervention**, producing c'_{transfer}.

Stage 3: Verifiable Pedagogical Reward & Post-Training (SFT + GRPO/DAPO). We first warm-start π_θ via Supervised Fine-Tuning (SFT) on 1,000 rejection-sampled Socratic trajectories. Next, we optimize π_θ via Group Relative Policy Optimization (GRPO) with dynamic sampling (**DAPO**) to filter out zero-variance prompts. For a tutor response a_t, the deterministic composite reward $R(a_t)$ is:

$$R(a_t) = \mathbb{I}_{\text{no-leak}}(a_t) \cdot \Big(\alpha\,\text{Pass}(c_{\text{final}}, U(P)) + \beta\,\Delta\text{Pass}(c'_{\text{transfer}}, U(P')) - \gamma\,\text{Tok}(a_t)\Big)$$

- **Executable Non-Leakage Gate ($\mathbb{I}_{\text{no-leak}}(a_t) \in \{0,1\}$):** A fast regex, Abstract Syntax Tree (AST), and lexical verifier that zeroes out the reward if a_t contains executable code blocks exceeding 2 lines, lacks diagnostic questioning, or exhibits $> 35\%$ n-gram overlap with the ground-truth patch. This prevents reward hacking via answer dumping or generic template injection.
- **Guided Resolution ($\text{Pass}(c_{\text{final}}, U(P)) \in [0,1]$):** The fraction of unit tests passed on practice problem P after tutor hint a_t.
- **Isomorphic Transfer Gain ($\Delta\text{Pass}(c'_{\text{transfer}}, U(P'))$):** The core pedagogical signal—measuring the improvement in π_{stu}'s unit-test pass rate on the unseen task P' conditioned on a_t, relative to its baseline pass rate without tutoring. This forces π_θ to explain generalizable *principles* ("why" the bug occurred) rather than localized edits ("what" to type).
- **Length Penalty ($\gamma\text{Tok}(a_t)$):** Penalizes overly verbose explanations to encourage concise Socratic guidance.

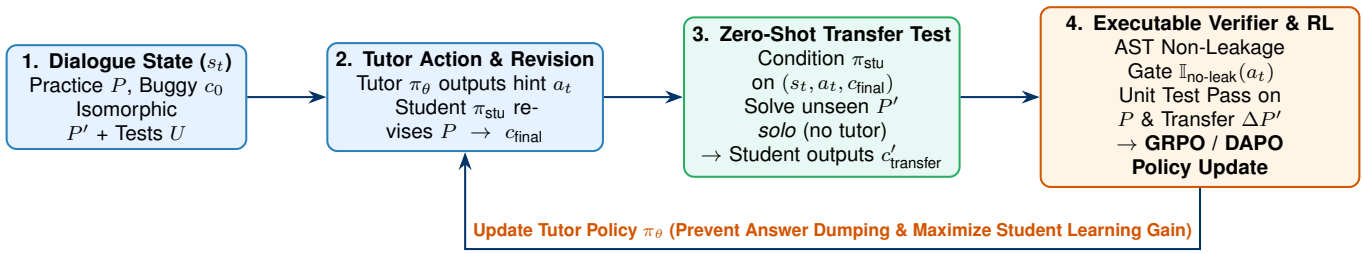


Figure 1: Overview of the **SocraticCoder-7B** training pipeline. Using State-Jump RL, Tutor π_θ generates a pedagogical intervention a_t for dialogue state s_t, and is updated via GRPO/DAPO using executable unit-test verification on both current problem P and a held-out isomorphic transfer task P'.

2.2 Evaluation Methods & Ablation Studies

We evaluate **SocraticCoder-7B** on the held-out test suite of 200 paired problems (P, P') across three dimensions (allowing full multi-turn rollouts $T \leq 4$ at test time):

1. **Compute-Matched Baselines:** Under an identical inference token budget, we compare against: (a) *Frontier Prompting* (`GPT-5` / `Claude 4 Sonnet` and `Kimi K2` with strict Socratic system prompts), (b) *Base Model + Best-of-N Filtering* (`Qwen2.5-Coder-7B` / `Qwen3-Coder` with heuristic leakage rejection), and (c) our *SFT-Only* checkpoint.
2. **Quantitative Metrics:** (i) **Transfer Learning Gain (ΔPass@1 on P')** evaluated across 2 distinct student models (`Llama-3.1-8B-Instruct` and `Qwen2.5-3B-Instruct`) to test cross-student generalization; (ii) **Adversarial Leakage Rate (%)** when the student prompt repeatedly demands direct code; and (iii) **Transfer Gain per 1k Tokens**.
3. **Ablations & Interactive Demo:** We conduct a clean ablation comparing $\beta = 0$ (rewarding only current problem P) vs. $\beta > 0$ (including transfer task P') to quantify how transfer rewards suppress subtle hint-leaking. By Week 6, we will deploy an interactive Gradio web IDE (side-by-side code editor + Socratic chat) and conduct a qualitative pilot with 5 Bootcamp peers.

2.3 Compute & Resource Requests

Table 1 details our resource allocation across the 6-week plan. By utilizing single-turn State-Jump rollouts powered by `vLLM` + `verl` and parallel `pytest` workers, our RL training fits comfortably within a modest 2x A100 budget.

Phase	Workload	Hrs
W1–2 Data/Env	1.2k paired (P, P') + pytest filter	20
W2–3 SFT	1k Socratic trajectories fine-tune	10
W3–4 RL	GRPO state-jump rollouts	50
W5–6 Eval	Ablations + multi-student + UI demo	20
API credits	Seed generation & baselines	$40
Total	**6-week lifecycle (2×A100 80G)**	**100**

Table 1: Compute & API budget breakdown.

3. Brief Literature Review

Pedagogical & Socratic LLMs: Prior benchmarks and tutoring models such as *MathDial* (Macina et al., 2023) and *SocraticLM* (Liu et al., 2024) rely primarily on Supervised Fine-Tuning (SFT) over static teacher-student transcripts. However, SFT-only models suffer from exposure bias and readily collapse into solution-dumping under student pressure. We advance this direction by introducing verifiable RL grounded in objective transfer execution. **Synthetic Data & Executable Checkers:** Recent frontier systems including **Kimi K2**, **GLM-4.5**, and **EigenData** highlight the effectiveness of pairing synthetic multi-turn interactions with deterministic executable checkers. We adapt executable verification from tool-use automation to human–AI pedagogy: using unit-test execution on a *secondary isomorphic problem* as an objective checker for conceptual teaching quality. **Reasoning RL & Scalable Rollouts:** Group-based RL algorithms such as **GRPO** (DeepSeek-R1) and **DAPO** (Yu et al., 2025), alongside asynchronous rollout engines like **slime** and **verl**, eliminate the memory overhead of critic networks while stabilizing optimization. We incorporate DAPO's dynamic sampling to filter out zero-variance states where the student solves P trivially. **Alignment & Preserving Human Agency:** Inspired by **Claude's Constitution** (*Teaching Why*) and emerging perspectives on human–AI complementarity, our framework operationalizes "helping humans learn rather than doing the work for them" into a scalable reinforcement learning objective, culminating in an interactive coding tutor demo by Week 6.

References

[1] Yu et al. (2025). *DAPO: An Open-Source LLM Reinforcement Learning System at Scale*. arXiv:2503.14476.

[2] Moonshot AI (2025). *Kimi K2: Open Agentic Intelligence & Synthetic Tool-Use Environments*. Technical Report.

[3] Liu et al. (2024). *SocraticLM: Exploring Socratic Personalized Teaching with LLMs*. NeurIPS 2024.

[4] DeepSeek-AI (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via RL*. arXiv:2501.12948.